

Principios de programación segura

1. Validación y gestión de errores

Tu proyecto, tal y como está ahora, no comprueba quién hace las peticiones ni les pone ningún límite: cualquiera que sepa la URL puede leer, crear, modificar o borrar lo que quiera. Antes de tocar autenticación, toca dar un paso atrás y preguntarse algo más básico: ¿qué significa exactamente que una aplicación sea "segura"?

Un caso concreto, sin atacante experto

- Manda un POST /api/v1/videojuegos con un precio negativo. @Valid rechaza la petición, pero la respuesta usa el formato genérico de Spring y no indica de forma clara qué campo ha fallado.
- Pide un listado ordenado por un campo que no existe, por ejemplo ?sort=noExiste,asc: la petición puede acabar en un 500, aunque el problema está realmente en el parámetro enviado por el cliente.

Ninguno de los dos casos necesita un atacante: basta con un cliente mal programado o con probar la API manualmente.

Qué significa que una aplicación sea "segura"

La seguridad de una aplicación se apoya, clásicamente, en tres pilares:

Pilar	En una aplicación web
Confidencialidad	Que nadie lea lo que no debe (datos de otro usuario, contraseñas...).
Integridad	Que nadie modifique lo que no debe (cambiar el precio de un producto ajeno, alterar un pedido).
Disponibilidad	Que el servicio siga funcionando (no se caiga por un ataque o un uso malintencionado).

El modelo mental del atacante

Ese mismo hueco, aceptar cualquier dato sin comprobarlo, es también la puerta favorita de quien sí actúa con mala intención. Toda API tiene una superficie de ataque: todo endpoint expuesto, todo dato de entrada que acepta, todo mensaje de error que devuelve — cualquier punto por el que algo externo puede interactuar con tu sistema es, potencialmente, un punto de ataque.

Nunca te fíes de lo que llega de fuera

Todo dato de entrada es hostil hasta que se valida — no porque asumas mala fe de cada usuario, sino porque no controlas qué envía realmente el cliente: puede ser un formulario mal rellenado, un bug del cliente, o alguien probando deliberadamente a romper tu API.

Principios generales de programación segura

- Validar en la frontera: comprobar los datos de entrada en el punto donde entran al sistema, antes de que lleguen a la lógica de negocio.
- Mínimo privilegio / mínima exposición: por defecto, todo cerrado; se abre explícitamente solo lo necesario.
- No filtrar información interna en los errores: un mensaje de error no debería revelar detalles de implementación.
- No guardar secretos en el código: contraseñas, claves, tokens viven en configuración externa, nunca en una clase Java.
- Defensa en profundidad: varias capas de protección, no una sola barrera — si una falla, las siguientes siguen ahí.

¿Y la contraseña de PostgreSQL en application-dev.yml?

En el entorno local de GameVault usamos una contraseña sencilla y versionada porque solo pertenece a una base de datos de desarrollo creada dentro de Docker y no se reutiliza en ningún entorno real.

Esto es aceptable para una credencial local y desechable, pero no para una base de datos desplegada ni para ningún secreto real. En esos casos, las credenciales deben quedar fuera del repositorio y proporcionarse mediante variables de entorno u otro sistema de configuración externa.

Un ataque real detrás de cada principio

Ataque	En qué consiste
Inyección	Código que trata datos externos como si fueran código — ya lo has visto de primera mano en Acceso a Datos, con JDBC puro y Statement.
Datos malformados	Entradas que rompen la lógica de la aplicación si no se comprueban antes de usarlas.
Fuerza bruta sobre logins	Probar contraseñas una tras otra hasta acertar.
Robo de credenciales en tránsito	Interceptar datos sensibles mientras viajan por la red.

Dos prácticas concretas, hoy

De los cinco principios, hoy conviertes dos en código real: validar en la frontera y no filtrar información en los errores. Los otros tres —mínima exposición, no guardar secretos, defensa en profundidad— los verás materializados en los próximos apartados, cuando entre en juego la autenticación.

Validación de entrada con Bean Validation

Las anotaciones de Bean Validation ya las conoces de Acceso a Datos. Lo que falta es decirles qué mensaje devolver cuando fallan, con el atributo message:

```
public record LibroCreatedDTO(  
    @NotBlank(message = "El título no puede estar vacío")  
    @Size(max = 150, message = "El título no puede superar los 150 caracteres")  
    String titulo,  
  
    @NotNull(message = "El precio es obligatorio")  
    @PositiveOrZero(message = "El precio no puede ser negativo")  
    BigDecimal precio  
    // ...  
) {}
```

@Valid: validar en la frontera, hecho código

El `create(...)` del controller ya tiene `@Valid` sobre este DTO, así que nada cambia en esa parte: `@Valid` activa la comprobación de estas anotaciones antes de que el cuerpo del método se ejecute — si algo no cumple, ni siquiera llega a tu lógica de negocio. Esto es "validar en la frontera" hecho código: el dato hostil se rechaza en la puerta, no dentro de la casa.

Gestión centralizada de errores

Sin nada más, Spring devuelve una respuesta por defecto — funcional, pero poco cuidada. La solución centraliza la conversión de excepciones en un único punto, con un formato común:

```
public record ErrorResponse(  
    String timestamp, int status, String error,  
    String message, String path,  
    Map<String, List<String>> fields  
) {  
    public ErrorResponse(String timestamp, int status,  
        String error, String message, String path) {  
        this(timestamp, status, error, message, path, Map.of());  
    }  
}
```

Qué guarda cada campo de ErrorResponse

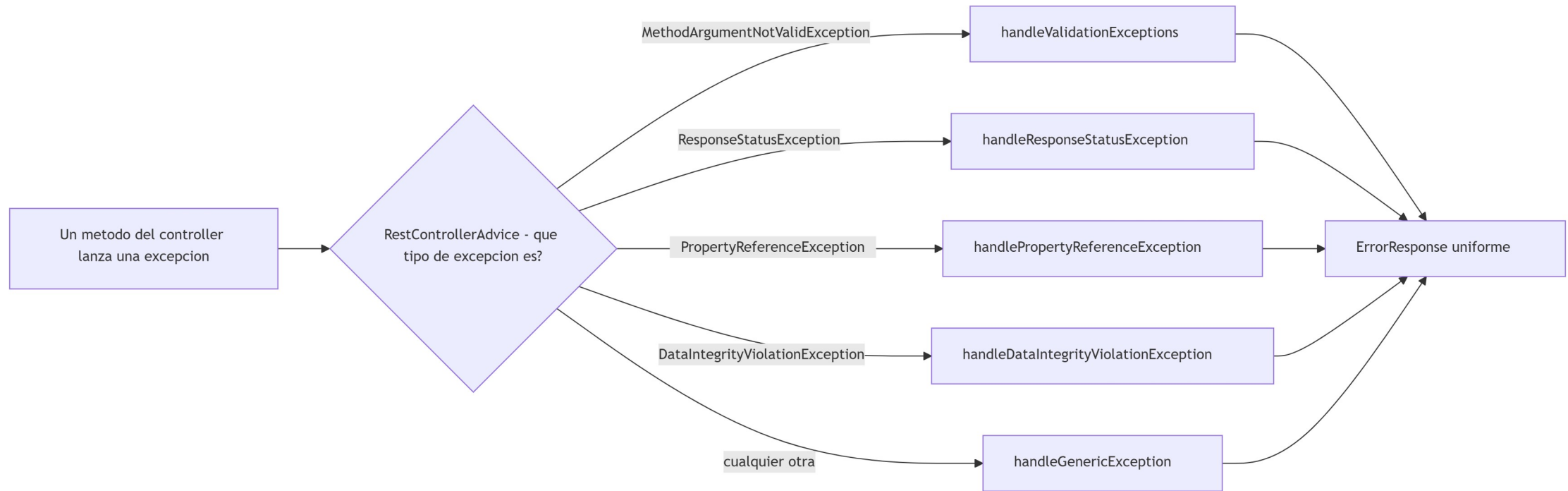
Campo	Qué guarda
timestamp	Cuándo ha ocurrido el error.
status	El código HTTP, repetido también en el cuerpo, no solo en la cabecera.
error	Nombre corto del error ("Error de validación", "Not Found"...).
message	Explicación legible, pensada para quien consume la API.
path	La ruta que se ha pedido, para saber qué endpoint ha fallado.
fields	Solo en errores de validación: qué campo ha fallado y con qué mensajes.

@RestControllerAdvice: un punto transversal

Con esa forma decidida, `@RestControllerAdvice` es la anotación que la pone en marcha de verdad: marca una clase como punto transversal, que intercepta las excepciones de todos los controllers de la aplicación, no de uno solo. Dentro, cada método marcado con `@ExceptionHandler(TipoDeExcepcion.class)` decide qué tipo de excepción atrapa y cómo la convierte en un `ErrorResponse`.

De la excepción al ErrorResponse uniforme

Vas a construir la clase con sus cinco handlers uno a uno — cada uno responde a una excepción que ya conoces de algún sitio:



Handler 1: la validación que acabas de ver

Atrapa lo que provoca `@Valid` cuando `LibroCreateDTO` no cumple alguna de sus anotaciones:

```
@ExceptionHandler(MethodArgumentNotValidException.class)
public ResponseEntity<ErrorResponse> handleValidationExceptions(
    MethodArgumentNotValidException ex, HttpServletRequest request) {

    Map<String, List<String>> fields = ex.getBindingResult().getFieldErrors()
        .stream().collect(Collectors.groupingBy(FieldError::getField,
            Collectors.mapping(FieldError::getDefaultMessage, Collectors.toList())));

    ErrorResponse response = new ErrorResponse(
        LocalDateTime.now().toString(), 400, "Error de validación",
        "La petición contiene campos inválidos", request.getRequestURI(), fields);
    return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(response);
}
```

Handler 1: qué hace cada pieza

`ex.getBindingResult().getFieldErrors()` es la lista de campos que han fallado la validación, con el mensaje que tú mismo has escrito en el DTO (`message = "..."`). `Collectors.groupingBy` la convierte en un mapa campo → lista de mensajes: si un mismo campo incumple dos anotaciones a la vez, aparecen los dos mensajes, no solo el primero.

Handler 2: los errores que ya lanzas tú

Atrapa `ResponseStatusException`, la misma que usas en el service con `orElseThrow(...)` — el mismo 404 que viste en el Tema 1:

```
@ExceptionHandler(ResponseStatusException.class)
public ResponseEntity<ErrorResponse> handleResponseStatusException(
    ResponseStatusException ex, HttpServletRequest request) {

    int statusCode = ex.getStatusCode().value();
    HttpStatus status = HttpStatus.resolve(statusCode);
    String error = status != null ? status.getReasonPhrase() : "Error";
    String message = ex.getReason() != null ? ex.getReason() : error;

    ErrorResponse response = new ErrorResponse(
        LocalDateTime.now().toString(), statusCode, error, message, request.getRequestURI());
    return ResponseEntity.status(ex.getStatusCode()).body(response);
}
```

Handler 2: el código de estado no está fijo

Aquí el código de estado no está fijo, al contrario que en el handler anterior (siempre 400): sale de la propia excepción, con `ex.getStatusCode()`, porque un `ResponseStatusException` puede llevar cualquier código que tú hayas decidido al lanzarla — no solo 404.

Handler 3: un parámetro que Spring Data no interpreta

Pedir un listado con `?sort=noExiste,asc` lanza `PropertyReferenceException`: Spring Data no encuentra esa propiedad en tu entidad. Sin capturarla, la petición puede acabar en un 500 aunque el fallo sea del cliente:

```
@ExceptionHandler(PropertyReferenceException.class)
public ResponseEntity<ErrorResponse> handlePropertyReferenceException(
    PropertyReferenceException ex, HttpServletRequest request) {

    ErrorResponse response = new ErrorResponse(
        LocalDateTime.now().toString(), 400, "Parámetro inválido",
        "Uno de los parámetros de la consulta no es válido", request.getRequestURI());
    return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(response);
}
```

Handler 3: por qué el mensaje es fijo

A diferencia del primer handler, aquí no hay un campo concreto que señalar —la excepción no identifica el parámetro que ha fallado con la misma precisión que `FieldError`—, así que el mensaje es fijo. Sigue siendo mucho mejor que devolver un 500 genérico o información interna de la aplicación.

Handler 4: una restricción que solo existe en la BD

Bean Validation no consulta la base de datos. Dos ejemplos de tu propio GameVault, con resultados distintos:

Operación	Qué ocurre
POST /videojuegos con un estudioId inexistente	VideojuegoService.create() busca primero el Estudio y lanza un 404 controlado si no existe.
DELETE /estudios/{id} con videojuegos asociados	Si el borrado no se propaga mediante cascade, PostgreSQL rechaza la operación por la clave foránea.

Handler 4: código y por qué 409 Conflict

```
@ExceptionHandler(DataIntegrityViolationException.class)
public ResponseEntity<ErrorResponse> handleDataIntegrityViolationException(
    DataIntegrityViolationException ex, HttpServletRequest request) {

    ErrorResponse response = new ErrorResponse(
        LocalDateTime.now().toString(), 409, "Conflicto de datos",
        "La operación viola una restricción de la base de datos (una referencia " +
        "que no existe, un registro del que aún dependen otros, un valor duplicado, etc.)",
        request.getRequestURI());
    return ResponseEntity.status(HttpStatus.CONFLICT).body(response);
}
```

409, no 400: la petición está bien construida, pero no puede completarse porque entra en conflicto con el estado actual de los datos.

Handler 5: la red de seguridad final

Cualquier otra excepción —un bug en tu propio código, por ejemplo— queda fuera del formato común. Un último handler genérico controla la respuesta:

```
@ExceptionHandler(Exception.class)
public ResponseEntity<ErrorResponse> handleGenericException(
    Exception ex, HttpServletRequest request) {

    ErrorResponse response = new ErrorResponse(
        LocalDateTime.now().toString(), 500, "Error interno",
        "Ha ocurrido un error inesperado", request.getRequestURI());
    return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(response);
}
```

⚠ **No devuelvas `ex.getMessage()` aquí**

Es tentador pasarle al cliente el mensaje real para depurar más rápido, pero puede contener detalles internos. Devuelve un mensaje genérico y registra la excepción completa en los logs.

Los cinco handlers, de un vistazo

Handler	Qué atrapa	Cuándo salta
handleValidationExceptions	MethodArgumentNotValidException	Un DTO con @Valid no cumple sus anotaciones
handleResponseStatusException	ResponseStatusException	Un orElseThrow(...) lanza un 404, u otro código explícito
handlePropertyReferenceException	PropertyReferenceException	Un sort sobre un campo que no existe
handleDataIntegrityViolationException	DataIntegrityViolationException	La base de datos rechaza la operación
handleGenericException	Exception	Cualquier otra excepción no prevista

El resto de este tema va a materializar, uno a uno, los principios que has visto hoy: mínima exposición se convierte en rutas cerradas por defecto ("Roles y rutas protegidas"), no guardar secretos en el código se convierte en el `jwt.secret` como propiedad externa ("Autenticación con JWT"), no almacenar contraseñas en claro se convierte en BCrypt ("Usuarios persistidos y BCrypt"). Hoy has visto los principios; en los próximos apartados les pones nombre y apellido en Spring Security.

Ideas clave

- Los tres pilares clásicos de la seguridad: confidencialidad, integridad, disponibilidad.
- La superficie de ataque es todo punto por el que algo externo interactúa con tu sistema; la regla de oro es no fiarse nunca de los datos de entrada.
- Principios clave: validar en la frontera, mínima exposición, no filtrar información en los errores, no guardar secretos en el código, defensa en profundidad.
- `@Valid` + anotaciones de `jakarta.validation` en los DTOs validan la entrada antes de que llegue a la lógica de negocio.

Ideas clave (cont.)

- `@RestControllerAdvice` + `@ExceptionHandler` centralizan la conversión de excepciones en respuestas HTTP coherentes, sin filtrar detalles internos.
- Cinco handlers cubren los casos conocidos: `MethodArgumentNotValidException` (validación), `ResponseStatusException` (404 y similares), `PropertyReferenceException` (parámetros que Spring Data no interpreta), `DataIntegrityViolationException` (restricciones de la base de datos) y, como red de seguridad final, `Exception` genérica.